

AUTOMATION

Run prompts on a schedule

Use `/loop` and the cron scheduling tools to run prompts repeatedly, poll for status, or set one-time reminders within a Claude Code session.

 Scheduled tasks require Claude Code v2.1.72 or later. Check your version with `claude --version`.

Scheduled tasks let Claude re-run a prompt automatically on an interval. Use them to poll a deployment, babysit a PR, check back on a long-running build, or remind yourself to do something later in the session. To react to events as they happen instead of polling, see **Channels**: your CI can push the failure into the session directly. To keep the session working turn after turn until a condition is met rather than on an interval, see `/goal`.

Tasks are session-scoped: they live in the current conversation and stop when you start a new one. Resuming with `--resume` or `--continue` brings back any task that hasn't **expired**: a recurring task created within the last 7 days, or a one-shot whose scheduled time hasn't passed yet. For scheduling that survives independently of any session, use **Routines** to create a routine on Anthropic-managed infrastructure, set up a **Desktop scheduled task**, or use **GitHub Actions**.

Compare scheduling options

Claude Code offers three ways to schedule recurring or one-off work:

	<u>Cloud</u>	<u>Desktop</u>	<u>/loop</u>
Runs on	Anthropic cloud	Your machine	Your machine
Requires machine on	No	Yes	Yes
Requires open session	No	No	Yes
Persistent across restarts	Yes	Yes	Restored on <code>--resume</code> if unexpired
Access to local files	No (fresh clone)	Yes	Yes

	<u>Cloud</u>	<u>Desktop</u>	<u>/loop</u>
MCP servers	Connectors configured per task	<u>Config files</u> and connectors	Inherits from session
Permission prompts	No (runs autonomously)	Configurable per task	Inherits from session
Customizable schedule	Via <code>/schedule</code> in the CLI	Yes	Yes
Minimum interval	1 hour	1 minute	1 minute

💡 Use **cloud tasks** for work that should run reliably without your machine. Use **Desktop tasks** when you need access to local files and tools. Use `/loop` for quick polling during a session.

Run a prompt repeatedly with /loop

The `/loop` **bundled skill** is the quickest way to run a prompt on repeat while the session stays open. Both the interval and the prompt are optional, and what you provide determines how the loop behaves.

What you provide	Example	What happens
Interval and prompt	<code>/loop 5m check the deploy</code>	Your prompt runs on a <u>fixed schedule</u>
Prompt only	<code>/loop check the deploy</code>	Your prompt runs at an <u>interval Claude chooses</u> each iteration
Interval only, or nothing	<code>/loop</code>	The <u>built-in maintenance prompt</u> runs, or your <code>loop.md</code> if one exists

You can also pass another command as the prompt, for example `/loop 20m /review-pr 1234`, to re-run a saved skill or command each iteration.

Run on a fixed interval

When you supply an interval, Claude converts it to a cron expression, schedules the job, and confirms the cadence and job ID.

```
/loop 5m check if the deployment finished and tell me what happened
```

The interval can lead the prompt as a bare token like `30m` , or trail it as a clause like `every 2 hours` . Supported units are `s` for seconds, `m` for minutes, `h` for hours, and `d` for days.

Seconds are rounded up to the nearest minute since cron has one-minute granularity. Intervals that don't map to a clean cron step, such as `7m` or `90m` , are rounded to the nearest interval that does and Claude tells you what it picked.

Let Claude choose the interval

When you omit the interval, Claude chooses one dynamically instead of running on a fixed cron schedule. After each iteration it picks a delay between one minute and one hour based on what it observed: short waits while a build is finishing or a PR is active, longer waits when nothing is pending. The chosen delay and the reason for it are printed at the end of each iteration.

The example below checks CI and review comments, with Claude waiting longer between iterations once the PR goes quiet:

```
/loop check whether CI passed and address any review comments
```

When you ask for a dynamic `/loop` schedule, Claude may use the **Monitor tool** directly. Monitor runs a background script and streams each output line back, which avoids polling altogether and is often more token-efficient and responsive than re-running a prompt on an interval.

A dynamically scheduled loop appears in your **scheduled task list** like any other task, so you can list or cancel it the same way. The **jitter rules** don't apply to it, but the **seven-day expiry** does: the loop ends automatically seven days after you start it.

❗ On Bedrock, Vertex AI, and Microsoft Foundry, a prompt with no interval runs on a fixed 10-minute schedule instead.

Run the built-in maintenance prompt

When you omit the prompt, Claude uses a built-in maintenance prompt instead of one you supply. On each iteration it works through the following, in order:

- continue any unfinished work from the conversation
- tend to the current branch's pull request: review comments, failed CI runs, merge conflicts

- run cleanup passes such as bug hunts or simplification when nothing else is pending

Claude does not start new initiatives outside that scope, and irreversible actions such as pushing or deleting only proceed when they continue something the transcript already authorized.

```
/loop
```

A bare `/loop` runs this prompt at a **dynamically chosen interval**. Add an interval, for example `/loop 15m`, to run it on a fixed schedule instead. To replace the built-in prompt with your own default, see **Customize the default prompt with loop.md**.

⚠ On Bedrock, Vertex AI, and Microsoft Foundry, `/loop` with no prompt prints the usage message instead of running the maintenance prompt.

Customize the default prompt with loop.md

A `loop.md` file replaces the built-in maintenance prompt with your own instructions. It defines a single default prompt for bare `/loop`, not a list of separate scheduled tasks, and is ignored whenever you supply a prompt on the command line. To schedule additional prompts alongside it, use `/loop <prompt>` or **ask Claude directly**.

Claude looks for the file in two locations and uses the first one it finds.

Path	Scope
<code>.claude/loop.md</code>	Project-level. Takes precedence when both files exist.
<code>~/.claude/loop.md</code>	User-level. Applies in any project that does not define its own.

The file is plain Markdown with no required structure. Write it as if you were typing the `/loop` prompt directly. The following example keeps a release branch healthy:

```
.claude/loop.md

Check the `release/next` PR. If CI is red, pull the failing job log,
diagnose, and push a minimal fix. If new review comments have arrived,
address each one and resolve the thread. If everything is green and
quiet, say so in one line.
```

Edits to `loop.md` take effect on the next iteration, so you can refine the instructions while a loop is running. When no `loop.md` exists in either location, the loop falls back to the built-in maintenance prompt. Keep the file concise: content beyond 25,000 bytes is truncated.

❗ On Bedrock, Vertex AI, and Microsoft Foundry, `loop.md` isn't read and `/loop` with no prompt prints the usage message instead.

Stop a loop

To stop a `/loop` while it is waiting for the next iteration, press `Esc`. This clears the pending wakeup so the loop does not fire again. Tasks you scheduled by asking Claude directly are not affected by `Esc` and stay in place until you delete them.

In self-paced mode, Claude can also end the loop on its own by not scheduling the next wakeup once the task is provably complete. Loops on a fixed interval keep running until you stop them or seven days elapse.

Set a one-time reminder

For one-shot reminders, describe what you want in natural language instead of using `/loop`. Claude schedules a single-fire task that deletes itself after running.

```
remind me at 3pm to push the release branch
```

```
in 45 minutes, check whether the integration tests passed
```

Claude pins the fire time to a specific minute and hour using a cron expression and confirms when it will fire.

Manage scheduled tasks

Ask Claude in natural language to list or cancel tasks, or reference the underlying tools directly.

```
what scheduled tasks do I have?
```

cancel the deploy check job

Under the hood, Claude uses these tools:

Tool	Purpose
<code>CronCreate</code>	Schedule a new task. Accepts a 5-field cron expression, the prompt to run, and whether it recurs or fires once.
<code>CronList</code>	List all scheduled tasks with their IDs, schedules, and prompts.
<code>CronDelete</code>	Cancel a task by ID.

Each scheduled task has an 8-character ID you can pass to `CronDelete`. A session can hold up to 50 scheduled tasks at once.

How scheduled tasks run

The scheduler checks every second for due tasks and enqueues them at low priority. A scheduled prompt fires between your turns, not while Claude is mid-response. If Claude is busy when a task comes due, the prompt waits until the current turn ends.

All times are interpreted in your local timezone. A cron expression like `0 9 * * *` means 9am wherever you're running Claude Code, not UTC.

Jitter

To avoid every session hitting the API at the same wall-clock moment, the scheduler adds a deterministic offset to fire times:

- Recurring tasks fire up to 30 minutes after the scheduled time (or up to half the interval, for tasks that run more often than hourly). An hourly job scheduled for `:00` may fire anywhere up to `:30`.
- One-shot tasks scheduled for the top or bottom of the hour fire up to 90 seconds early.

The offset is derived from the task ID, so the same task always gets the same offset. If exact timing matters, pick a minute that is not `:00` or `:30`, for example `3 9 * * *` instead of `0 9 * * *`, and the one-shot jitter will not apply.

Seven-day expiry

Recurring tasks automatically expire 7 days after creation. The task fires one final time, then deletes itself. This bounds how long a forgotten loop can run. If you need a recurring task to last longer, cancel and recreate it before it expires, or use [Routines](#) or [Desktop scheduled tasks](#) for durable scheduling.

Cron expression reference

`CronCreate` accepts standard 5-field cron expressions: `minute hour day-of-month month day-of-week`. All fields support wildcards (`*`), single values (`5`), steps (`*/15`), ranges (`1-5`), and comma-separated lists (`1,15,30`).

Example	Meaning
<code>*/5 * * * *</code>	Every 5 minutes
<code>0 * * * *</code>	Every hour on the hour
<code>7 * * * *</code>	Every hour at 7 minutes past
<code>0 9 * * *</code>	Every day at 9am local
<code>0 9 * * 1-5</code>	Weekdays at 9am local
<code>30 14 15 3 *</code>	March 15 at 2:30pm local

Day-of-week uses `0` or `7` for Sunday through `6` for Saturday. Extended syntax like `L` , `W` , `?` , and name aliases such as `MON` or `JAN` is not supported.

When both day-of-month and day-of-week are constrained, a date matches if either field matches. This follows standard vixie-cron semantics.

Disable scheduled tasks

Set `CLAUDE_CODE_DISABLE_CRON=1` in your environment to disable the scheduler entirely. The cron tools and `/loop` become unavailable, and any already-scheduled tasks stop firing. See [Environment variables](#) for the full list of disable flags.

Limitations

Session-scoped scheduling has inherent constraints:

- Tasks only fire while Claude Code is running and idle. Closing the terminal or letting the session exit stops them firing.
- No catch-up for missed fires. If a task's scheduled time passes while Claude is busy on a long-running request, it fires once when Claude becomes idle, not once per missed interval.
- Starting a fresh conversation clears all session-scoped tasks. Resuming with `claude --resume` or `claude --continue` restores tasks that have not expired: recurring tasks within seven days of creation, and one-shot tasks whose scheduled time has not yet passed. Background Bash and monitor tasks are never restored on resume.

For cron-driven automation that needs to run unattended:

- **Routines**: run on Anthropic-managed infrastructure on a schedule, via API call, or on GitHub events
- **GitHub Actions**: use a `schedule` trigger in CI
- **Desktop scheduled tasks**: run locally on your machine